Projet master ISF: Assistant d’onboarding RH intelligent

Contexte

Vous travaillez pour une entreprise fictive. Votre objectif est de concevoir un **assistant conversationnel** capable d’accompagner un nouvel employé pendant sa **première semaine**.

Contrairement à un simple chatbot, l’assistant doit :

- **Répondre à des questions** en s’appuyant sur une base de connaissances interne.
- **Accéder à des données structurées** via une interface standard.
- **Déclencher des actions** au moyen d’outils (tools).

Objectif général

Développer un assistant qui, à partir d’une requête en langage naturel, est capable de :

1. Retrouver des informations RH (mutuelle, horaires, télétravail).
2. Identifier le manager du nouvel arrivant à partir d’une base “employés”.
3. Réaliser des actions simulées (message, réunion).

Fonctionnalités attendues

1) Module RAG: Base de connaissances

Mettre en place un module **RAG** permettant à l’assistant de répondre à des questions sur une **politique interne** fournie sous forme de documents.

Jeu de documents minimal (à fournir / générer) :

- Mutuelle
- Horaires
- Télétravail

2) Module Data: Accès à la base employés

Mettre en place une petite “base de données employés” (au choix **JSON** ou **SQLite**).

Champs attendus :

- Nom
- Rôle
- Email
- Manager

Créer un **sub-agent “DB Agent”** dont le rôle est d’exposer des tools d’accès aux données (exemples) :

- `get_employee(name: str)`
- `get_manager(employee_name: str)`
- `search_employees(query: str)`

Le *main agent* doit appeler ce sub-agent pour identifier **qui est le manager** du nouvel arrivant.

3) Module Actions: Tools via des *sub-agents*

Créer deux sub-agents dédiés aux actions, chacun exposant des tools :

A) Sub-agent “Message Agent” (simulation Slack)

- Tool `send_message(to: str, message: str)`
- Sortie attendue : accusé d’envoi simulé + log (ex: id fictif)

B) Sub-agent “Meeting Agent” (simulation calendrier)

- Tool `check_availability(attendees: list[str], start: str, duration_min: int)`
  - Sortie attendue : créneaux possibles (simulés) + log
- Tool `schedule_meeting(attendees: list[str], title: str, start: str, duration_min: int)`
  - Sortie attendue : événement simulé + log (ex: id fictif)

Le *main agent* doit orchestrer ces appels en fonction de la requête utilisateur (raisonnement + appels de tools).

Livrable

Un projet **Python** comprenant :

- le code (assistant + RAG + tools),
- une petite application **Gradio** / **Streamlit** pour interagir avec l'assistant.

Exemple de prompts à supporter :

> « Je viens d'arriver. Qui est mon manager et peux-tu lui envoyer un message pour lui dire que je suis prêt ? Ah, et comment fonctionne la mutuelle ? »

> « Peux-tu me rappeler les horaires de travail et vérifier si mon manager est disponible demain à 10h pour un point de 30 minutes ? »

> « Je cherche le contact RH pour le télétravail. Et peux-tu planifier une réunion d'onboarding avec mon manager et moi cette semaine ? »

Comportement attendu de l'agent :

1. Identifier le manager via le **DB Agent** (tools d'accès à la base employés).
2. Envoyer des messages via le **Message Agent** (tool `send_message`).
3. Vérifier les disponibilités via le **Meeting Agent** (tool `check_availability`).
4. Planifier des réunions via le **Meeting Agent** (tool `schedule_meeting`).
5. Répondre aux questions sur la mutuelle, les horaires, le télétravail et les contacts RH via **RAG**.
6. Rechercher des informations spécifiques sur les employés via le **DB Agent** (tool `search_employees`).

Critères d'évaluation

- Qualité de la conception de l'agent : Comment avez-vous pensé l'architecture globale ? Quels choix de design avez-vous faits et pourquoi ?
- Difficultés rencontrées : Quels obstacles avez-vous rencontrés lors du développement ? Comment les avez-vous surmontés ?
- Pertinence des réponses : L'agent répond-il de manière appropriée aux questions liées au sujet ? Fournit-il des citations ou justifications des sources (RAG) ?

- Gestion des questions hors-sujet : L'agent refuse-t-il poliment de répondre aux questions non liées au contexte RH et onboarding ?
- Qualité de l'accès aux données via le **DB Agent** : Les tools sont-ils bien conçus et utilisés à bon escient ?
- Qualité de la conception des sub-agents et des tools : Les sub-agents sont-ils bien définis et leurs tools correctement implémentés ?

💡 **Soutenance (attendu)** : votre application doit proposer un mode “debug / trace” permettant de comprendre **quels sub-agents et quels tools** ont été exécutés pour produire la réponse, ainsi que **quelles sources** ont été consultées pour le RAG.
Exemples de traces attendues (format libre) :

- Ordre des appels : DB Agent → get_manager, Meeting Agent → check_availability → schedule_meeting, Message Agent → send_message
- Résultat synthétique de chaque tool
- Documents/chunks consultés côté RAG (titres ou ids)